Advanced C# Concepts Report

Name: Asser Mohammed Khairallah
Group Code: GIZ5_SWD5_S2
Assignment: Advanced C# Concepts (Implicit Typing, Anonymous Types, and Delegates)

Part 01: Short Write-Ups (Theoretical Questions)

1. In your own words, what is the difference between a delegate and a lambda expression?

- A delegate is a strongly-typed reference (or pointer) that defines a specific method signature and return type; it acts as a contract. A lambda expression, on the other hand, is a concise, inline anonymous function used to provide the actual implementation for that delegate. In short, the delegate is the type or container, while the lambda expression is the value or logic assigned to it.

2. Why is ‘var’ still statically typed even though the type isn’t written explicitly?

- The `var` keyword is statically typed because the C# compiler infers and strictly defines the exact data type at compile-time based on the initialized value on the right-hand side of the assignment. Once the compiler determines this type, it is locked and cannot be changed at runtime (unlike dynamic typing). It is simply syntactic sugar that maintains C#’s strict type safety while reducing redundant code verbosity.

3. Give one real scenario where an anonymous type is more convenient than creating a full class.

- A highly practical scenario is when projecting specific data using LINQ and Entity Framework. If you are querying a large ‘Employees’ database table with 40 columns but only need the ‘Id’ and ‘FullName’ to populate a temporary UI dropdown menu, creating a dedicated Data Transfer Object (DTO) class just for this single query adds unnecessary boilerplate. Using an anonymous type (e.g., `.Select(e => new { e.Id, e.FullName })`) handles this elegantly, saving time and keeping the structure strictly scoped to where it is needed.